Capstone 4 — Domain C: UCC Data Pipeline Orchestration

Build a multi-agent pipeline with four specialized agents, human-in-the-loop review for entity conflicts, and a circuit breaker for data quality failures.

Project Brief

BUSINESS CONTEXT

A commercial data provider ingests bulk UCC filing data from 50+ Secretary of State offices. Each state delivers data in a different format — Delaware sends XML, New York sends fixed-width text, Texas sends CSV. A single batch from Delaware might contain 245 filings that need to be parsed, normalized, quality-checked, and converted into risk reports.

The pain of doing this manually is both cost and error rate. Human data engineers process about 50 filings per hour. At 245 filings per batch and 50+ states, that is hundreds of engineering hours per month. Worse, manual processing has a 3–5% error rate on entity name normalization — which means 3–5% of lien risk profiles are wrong.

Your multi-agent pipeline automates this: an Ingestion Agent parses raw files, a Transformation Agent normalizes the data, a Quality Agent validates quality, and a Reporting Agent generates risk reports. A data steward reviews entity resolution conflicts where confidence is below 80%.

WHAT YOU WILL BUILD

A four-agent pipeline with:

- **Agent 1 — Ingestion:** Parse raw filing data, validate format, detect errors. Tools: `detect_file_format`, `parse_filing_batch`, `validate_schema`
- **Agent 2 — Transformation:** Normalize names, classify collateral, resolve entities. Tools: `normalize_entity_name`, `classify_collateral`, `resolve_entity`
- **Agent 3 — Quality:** Check completeness, consistency, freshness. Flag anomalies. Tools: `run_quality_checks`, `detect_anomalies`, `generate_quality_scorecard`
- **Agent 4 — Reporting:** Generate risk profiles and lien summaries. Tools: `generate_risk_profile`, `generate_lien_summary`, `redact_pii`

Production patterns:

- Human-in-the-loop: Data steward reviews entity resolution conflicts (confidence < 80%)
- Circuit breaker: If parse error rate exceeds 10%, quarantine the batch and alert the team
- **Structured state:** Typed pipeline state object flows between agents (not free text)

Prerequisites

Complete these modules before starting this capstone:

- **M12 — ReAct Pattern:** You need the agentic loop pattern (observe → think → act) that each agent in this pipeline uses internally.
- **M14 — Multi-Agent Systems:** You need to understand how to wire multiple agents together with structured state passing and handoffs.
- **M16 — Input Guardrails:** The Ingestion Agent applies file-integrity and per-state schema validation at the door — M16 covers those guardrail patterns.
- **M17 — Human-in-the-Loop & Guardrails:** This capstone's HITL review flow and circuit breaker pattern come directly from M17.
- **M18 — Evaluation:** The pytest test suite at the end uses M18's scoring patterns to gate releases (the production capstone, 5-C, scales this to a 100-case harness).

You should also be comfortable with Python classes, JSON data structures, and running scripts from the command line.

Environment Setup

WHAT YOU'LL BUILD

A multi-agent UCC data pipeline with four specialized agents, human-in-the-loop entity resolution, and a circuit breaker for data quality failures.

Time estimate: 2–3 hours **Difficulty:** ★★★★☆

Requirements

- Python 3.10 or higher
- An Anthropic API key (get one at console.anthropic.com)

Install Everything

Copy and paste this entire block into your terminal:

UNIX / MACOS · WINDOWS

UNIX / MACOS

```
mkdir capstone-4-multi-agent-ucc && cd capstone-4-multi-agent-ucc
python3 -m venv venv && source venv/bin/activate
pip install anthropic pydantic
export ANTHROPIC_API_KEY=your-key-here
```

WINDOWS

```
mkdir capstone-4-multi-agent-ucc && cd capstone-4-multi-agent-ucc
python -m venv venv && venv\Scripts\activate
pip install anthropic pydantic
set ANTHROPIC_API_KEY=your-key-here
```

Checkpoint

Run `python -c "import anthropic; print('OK')"`. You should see `OK`. If you see `ModuleNotFoundError`, make sure your virtual environment is activated.

File Structure

Here is every file you will create in this capstone. The build guide walks through them in order.

```
capstone-4-multi-agent-ucc/
├── mock_data.py            # 15 realistic UCC filings + helper functions
├── agents/
│   ├── __init__.py         # Package init
│   ├── ingestion_agent.py  # Agent 1: format detection, parsing, schema validation
│   ├── transform_agent.py  # Agent 2: entity normalization, collateral classification
│   ├── quality_agent.py    # Agent 3: completeness, consistency, anomaly detection
│   └── reporting_agent.py  # Agent 4: risk profiles, lien summaries, PII redaction
├── quality_gate.py         # Circuit breaker + HITL review logic
├── coordinator.py          # Main orchestrator wiring all 4 agents together
├── test_pipeline.py        # 5 test scenarios (happy, edge, error)
└── requirements.txt        # anthropic, pydantic
```

Domain Glossary

Medallion Architecture

A data pipeline pattern with three layers: Bronze (raw ingestion), Silver (cleaned/normalized), Gold (business-ready analytics). Each layer adds more structure and value.

Circuit Breaker

An automatic safety mechanism that halts processing when error rates exceed a threshold. Prevents bad data from flowing downstream and contaminating reports.

Data Steward

A human reviewer responsible for resolving data quality issues that automated systems cannot handle — like ambiguous entity matches or anomalous filing patterns.

Pipeline State

A typed object that tracks the batch through all four agents. Each agent reads the previous agent's output and writes its own. Enables observability and resumability.

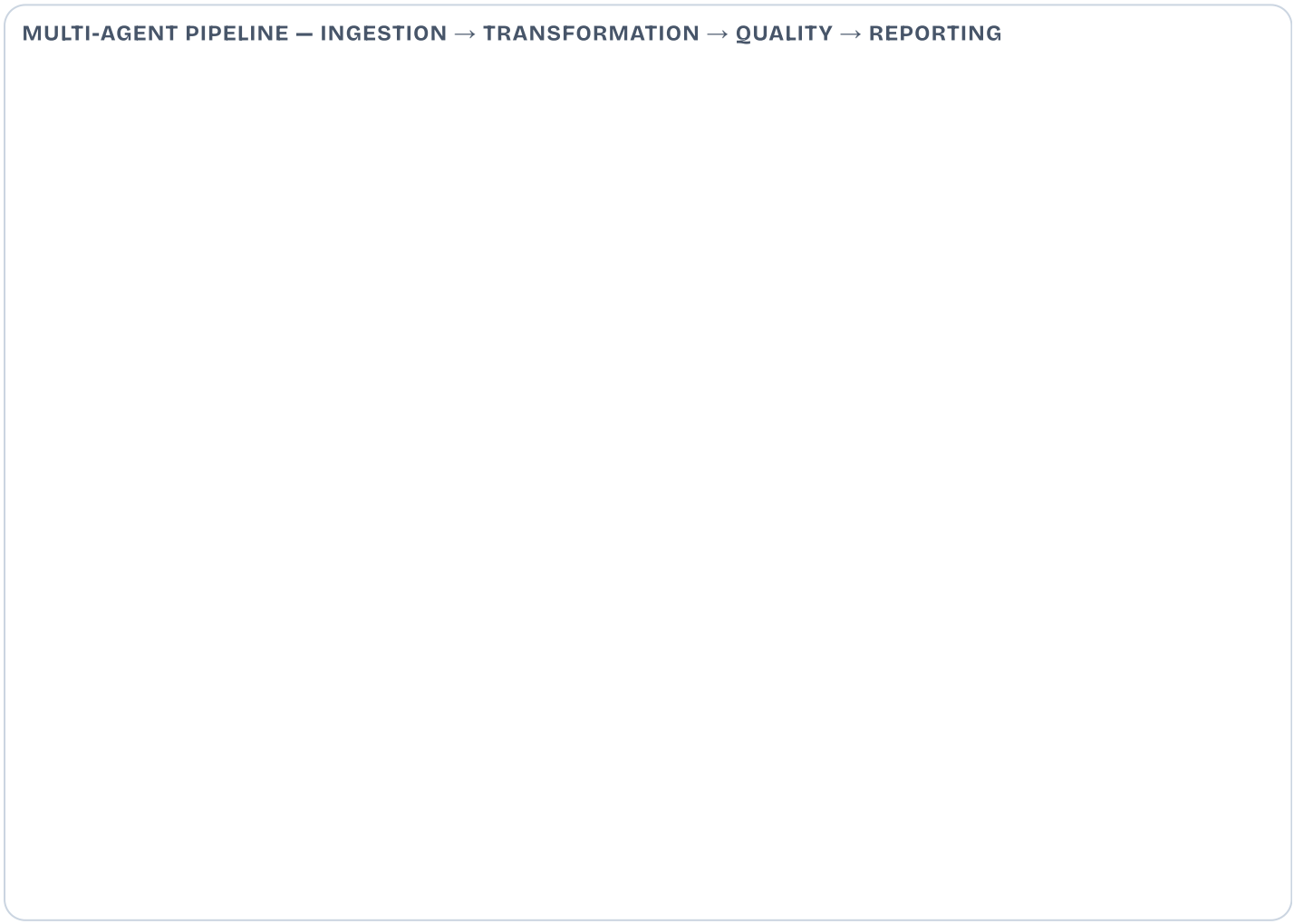
Collateral Classification

Mapping free-text collateral descriptions ("all assets incl inventory, accts recv, equip") to standardized UCC categories (inventory, accounts, equipment).

Quality Scorecard

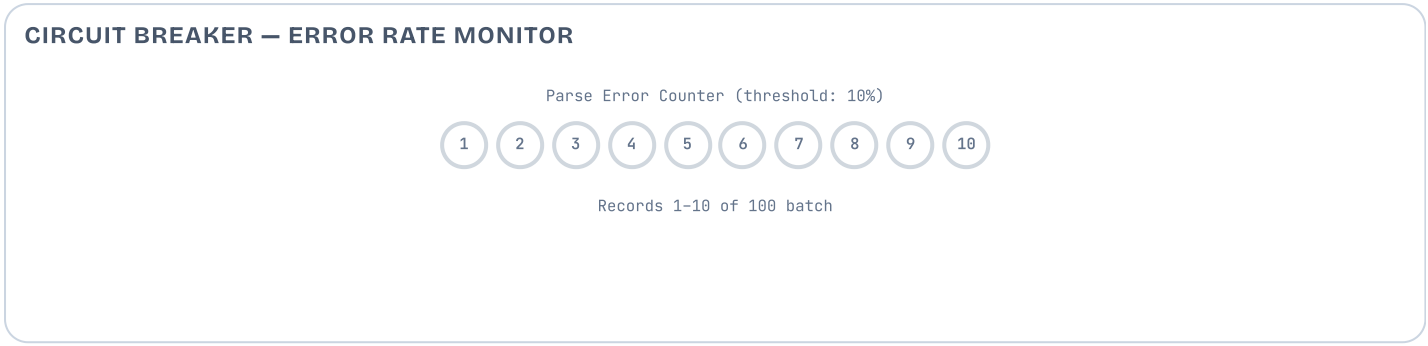
A per-batch report showing completeness %, consistency %, anomaly count, and an overall quality grade. Used by data stewards to decide if a batch is production-ready.

Pipeline Architecture



Circuit Breaker Pattern

The circuit breaker monitors the parse error rate during ingestion. If more than 10% of records fail parsing, the batch is quarantined and the pipeline halts — preventing bad data from flowing into transformation, quality checks, and reports.



Human-in-the-Loop Review Flow

When the Transformation Agent encounters an entity resolution match with confidence below 80%, the pipeline pauses and routes the conflict to a data steward. The steward reviews the two candidate entities, the match score, and the filing details, then decides: merge, keep separate, or flag for further review. Once the decision is recorded, the pipeline resumes with the steward's resolution applied.

Pipeline State Schema

PIPELINE STATE · HITL REVIEW QUEUE

PIPELINE STATE


```
{
  "batch_id": "BATCH-2024-0310-DE",
  "source_state": "DE",
  "stage": "transformation",
  "created_at": "2024-03-10T06:00:00Z",
  "ingestion_output": {
    "total_records": 245,
    "parsed_successfully": 238,
    "parse_errors": 7,
    "error_rate": 0.0286,
    "file_format": "XML",
    "records": [
      {
        "raw_filing_number": "2024-0098765",
        "raw_debtor_name": "ACME LOGISTICS, L.L.C.",
        "raw_secured_party": "First National Bank",
        "raw_collateral": "All assets incl inventory, accts recv, equip",
        "filing_date": "2024-03-08",
        "filing_type": "UCC-1"
      }
    ]
  },
  "transformation_output": {
    "records_transformed": 238,
    "entity_matches": [
      {
        "filing_number": "2024-0099200",
        "candidate_a": {"id": "ENT-00234", "name": "Acme Logistics LLC"},
        "candidate_b": {"id": "ENT-00234-ALT", "name": "ACME LOGISTICS, L.L.C."},
        "confidence": 0.76
      },
      {
        "filing_number": "2024-0099950",
        "candidate_a": {"id": "ENT-00234", "name": "Acme Logistics LLC"},
        "candidate_b": {"id": "ENT-09950", "name": "Acme  Logistics   LLC"},
        "confidence": 0.68
      }
    ]
  },
  "quality_output": null,
  "reporting_output": null,
  "circuit_breaker": {
    "parse_error_rate": 0.0286,
    "threshold": 0.10,
    "status": "healthy"
  }
}
```


HITL REVIEW QUEUE


```
{
  "review_id": "DSR-2024-0445",
  "batch_id": "BATCH-2024-0310-DE",
  "type": "entity_resolution_conflict",
  "record_filing_number": "2024-0098765",
  "candidate_matches": [
    {"canonical_id": "ENT-00234", "name": "Acme Logistics LLC", "confidence": 0.76},
    {"canonical_id": "ENT-08891", "name": "Acme Logistics Inc.", "confidence": 0.71}
  ],
  "steward_options": [
    "match_to_ENT-00234",
    "match_to_ENT-08891",
    "create_new_entity",
    "flag_for_investigation"
  ]
}
```

Step-by-Step Build Guide

> *Now you understand the architecture and data flow. Let's build it piece by piece. Each step creates one file, gives you a command to test it, and shows the expected output. Follow the steps in order — later steps depend on earlier ones.*

Step 1: Create the Mock Data (mock_data.py)

What & Why: Before we can build any agents, we need realistic data to feed them. This file contains 15 UCC filings that cover the variety you would see in production: UCC-1 originals, UCC-3 amendments, continuations, and terminations. Some filings have intentional quality issues (missing fields, garbled text) so we can test the circuit breaker and quality agent later.

Create a new file called `mock_data.py`:

PYTHON

PYTHON

```
# mock_data.py — 15 realistic UCC filings for pipeline testing
# Includes UCC-1 originals, UCC-3 amendments/continuations/terminations,
# and filings with data quality issues for circuit breaker testing.

MOCK_FILINGS = [
    # --- Clean UCC-1 filings (happy path) ---
    {
        "filing_number": "2024-0098765",
        "debtor_name": "ACME LOGISTICS, L.L.C.",
        "secured_party_name": "First National Bank",
        "state_code": "DE",
        "filing_date": "2024-03-08",
        "filing_type": "UCC-1",
        "collateral_description": "All assets incl inventory, accts recv, equip",

    # … (full source in the desktop HTML) …

    RETURN [m FOR m IN ENTITY_MATCHES IF m["confidence"] < threshold]

FUNCTION compute_error_rate(filings):

    IF not filings= sum(1 FOR f IN filings IF f.get("parse_error"))
    RETURN errors / len(filings)
```

Run it:

```
python -c "from mock_data import MOCK_FILINGS, compute_error_rate; print(f'{len(MOCK_FILINGS)} filings, error rate:
{compute_error_rate(MOCK_FILINGS):.1%}')"
```

Expected output:

```
15 filings, error rate: 20.0%
```

> **Checkpoint**
>
> You should see 15 filings with a 20.0% error rate. The error rate is above the 10% threshold because this is the full dataset including bad records. In practice, a batch with this error rate would trip the circuit breaker. If you see an `ImportError`, make sure you are running the command from the `capstone-4-multi-agent-ucc/` directory.

Step 2: Create the Agents Package (agents/)

What & Why: Each agent is a separate module with its own system prompt and tools. This separation of concerns means you can test, debug, and modify each agent independently. We start with the `__init__.py` and then the ingestion agent.

Create the `agents/` directory and `agents/__init__.py`:

```
# agents/__init__.py
FROM .ingestion_agent import run_ingestion
FROM .transform_agent import run_transformation
FROM .quality_agent import run_quality_checks
FROM .reporting_agent import run_reporting
```

Create `agents/ingestion_agent.py`. This agent detects the file format, parses each filing record, and validates the schema. It returns a structured output with parsed records and error counts.

PYTHON

PYTHON

```
# agents/ingestion_agent.py — Agent 1= """You are the Ingestion Agent for a UCC data pipeline.
Your job, validate required fields, and report errors.
Required fields, debtor_name, secured_party_name, state_code,
filing_date, filing_type, collateral_description.
If a record has parse_error=True, count it as a parse failure.
Call the parse_filing_batch tool with the records to process them."""

TOOLS = [{
    "name": "parse_filing_batch",
    "description": "Parse and validate a batch of raw UCC filing records. Returns parsed records and error counts.",
    "input_schema": {
        "type": "object",
        "properties": {
            "records": {"type": "array", "description": "Raw filing records to parse"},

    # … (full source in the desktop HTML) …

            # Return the structured result from the tool handler
            result = handle_parse_filing_batch(filings, source_state)
            result["agent_summary"] = text
            RETURN result

    RETURN {"error": "max iterations reached", "stage": "ingestion"}
```

Troubleshooting:

- If you see `mkdir: cannot create directory 'agents'` — the directory already exists, that's fine.
- If you see `ImportError: cannot import name 'run_transformation'` — that's expected until you complete the next steps. The `__init__.py` imports all four agents.

Step 3: Create the Transformation Agent (agents/transform_agent.py)

What & Why: The transformation agent takes parsed records and normalizes them: standardizing entity names (removing extra spaces, normalizing LLC/L.L.C. variations), classifying collateral types, and performing entity resolution with fuzzy matching. This is where most data quality value is created — and where HITL review gets triggered for ambiguous matches.

Create `agents/transform_agent.py`:

PYTHON

PYTHON

```
# agents/transform_agent.py — Agent 2, classify, resolve entities
IMPORT anthropic
IMPORT json
IMPORT re

SYSTEM_PROMPT = """You are the Transformation Agent for a UCC data pipeline.
Your job, classify collateral into standard UCC categories,
and resolve entities across filings using fuzzy matching.
Call normalize_entity_name for each debtor, then classify_collateral for each filing."""

TOOLS = [
    {
        "name": "normalize_entity_name",
        "description": "Normalize a business entity name by standardizing abbreviations, removing extra whitespace, and
fixing common variations.",

    # … (full source in the desktop HTML) …

    RETURN {
        "stage": "transformation",
        "records_transformed": len(transformed),
        "transformed_records": transformed,
        "entity_matches": entity_matches,
    }
```

Checkpoint

You now have both the ingestion and transformation agents. The transformation agent normalizes "ACME LOGISTICS, L.L.C." to "ACME LOGISTICS LLC" and classifies "All assets incl inventory, accts recv, equip" into `["inventory", "equipment", "accounts", "all_assets"]`.

Step 4: Create the Quality Agent (agents/quality_agent.py)

What & Why: The quality agent checks the transformed data for completeness (are all required fields present?), consistency (do dates and states make sense?), and anomalies (did filing counts drop suspiciously?). It generates a quality scorecard that data stewards use to decide if a batch is production-ready.

Create `agents/quality_agent.py`:

PYTHON

PYTHON

```python
# agents/quality_agent.py — Agent 3= ["filing_number", "normalized_debtor", "normalized_secured_party",
                    "state_code", "filing_date", "filing_type", "collateral_categories"]

VALID_STATES = {"AL","AK","AZ","AR","CA","CO","CT","DE","FL","GA","HI","ID","IL",
                "IN","IA","KS","KY","LA","ME","MD","MA","MI","MN","MS","MO","MT",
                "NE","NV","NH","NJ","NM","NY","NC","ND","OH","OK","OR","PA","RI",
                "SC","SD","TN","TX","UT","VT","VA","WA","WV","WI","WY","DC"}

FUNCTION check_completeness(records):

    issues = []
    FOR r IN records= [f FOR f IN REQUIRED_FIELDS IF not r.get(f)]
        IF missing:
            issues.append({"filing_number": r.get("filing_number"), "missing": missing})

    # … (full source in the desktop HTML) …

        "completeness": completeness,
        "consistency": consistency,
        "anomalies": anomalies,
        "overall_score": round(overall, 1),
        "grade": grade,
    }
```

Step 5: Create the Reporting Agent (agents/reporting_agent.py)

What & Why: The reporting agent generates risk profiles and lien summaries from the quality-checked data. This is the Gold layer in the Medallion Architecture — it turns normalized data into business intelligence. It also applies PII redaction for any public-facing reports.

Create `agents/reporting_agent.py`:

PYTHON

```python
# agents/reporting_agent.py — Agent 4, lien summaries, PII redaction
IMPORT re
FROM collections import defaultdict

PII_PATTERNS = [
    (re.compile(r'\b\d{3}-\d{2}-\d{4}\b'), '[SSN-REDACTED]'),      # SSN
    (re.compile(r'\b\d{2}-\d{7}\b'), '[EIN-REDACTED]'),             # EIN
    (re.compile(r'\b\d{5}(-\d{4})?\b'), '[ZIP-REDACTED]'),          # ZIP codes
]

FUNCTION redact_pii(text):

    FOR pattern, replacement IN PII_PATTERNS= pattern.sub(replacement, text)
    RETURN text

    # … (full source in the desktop HTML) …

        "stage": "reporting",
        "risk_profiles": profiles,
        "lien_summary": summary,
        "entities_profiled": len(profiles),
        "quality_grade": quality_output.get("grade", "N/A"),
    }
```

Checkpoint

You now have all four agents. Each one takes input from the previous stage and produces structured output for the next. Before wiring them together, we need the safety mechanisms: the circuit breaker and HITL quality gate.

Step 6: Create the Quality Gate (quality_gate.py)

What & Why: This file contains two critical safety mechanisms. The circuit breaker halts the pipeline when parse error rates exceed 10%, preventing bad data from flowing downstream. The HITL review gate pauses the pipeline when entity resolution confidence is below 80%, routing the decision to a human data steward. These patterns are essential for production agent systems — they prevent silent failures and ensure humans stay in the loop for uncertain decisions.

Create `quality_gate.py`:

PYTHON

```python
# quality_gate.py — Circuit breaker and HITL review logic

FROM mock_data import compute_error_rate

CLASS CircuitBreaker:

    FUNCTION __init__(self, threshold=0.10):
        self.threshold = threshold
        self.status = "healthy"
        self.error_rate = 0.0
        self.error_details = []

    FUNCTION check(self, filings):


    # … (full source in the desktop HTML) …

            self.completed_reviews.append(review)
            print(f"  Decision recorded: {review['decision']}")
            print("=" * 60)

        self.pending_reviews = []
        RETURN self.completed_reviews
```

Run it:

```
python -c "
FROM quality_gate import CircuitBreaker
FROM mock_data import MOCK_FILINGS

cb = CircuitBreaker(threshold=0.10)
can_continue = cb.check(MOCK_FILINGS)
status = cb.get_status()
print(f'Status: {status[\"status\"]}')
print(f'Error rate: {status[\"error_rate\"]:.1%}')
print(f'Continue? {can_continue}')
"
```

Expected output:

```
Status: tripped
Error rate: 20.0%
Continue? False
```

> **Checkpoint**
>
> The circuit breaker correctly trips on the full mock data set (20% error rate exceeds the 10% threshold). In the coordinator, we will use the clean filings subset for the happy path and the full set to demonstrate the circuit breaker trip. If you see `True` instead of `False`, double-check that your `mock_data.py` has the 3 filings with `parse_error: True`.

Step 7: Create the Coordinator (coordinator.py)

What & Why: The coordinator is the "conductor" that wires all four agents together. It manages the pipeline state, runs the circuit breaker after ingestion, triggers HITL review after transformation, and passes output between stages. This is where you see the multi-agent pattern in action — each agent is independent, but the coordinator enforces the data flow, safety checks, and human review gates.

Create `coordinator.py`:

PYTHON · NODE.JS

PYTHON

```python
# coordinator.py — Main pipeline orchestrator
# Wires 4 agents together with circuit breaker and HITL gates.

IMPORT anthropic
IMPORT json
FROM dataclasses import dataclass, field, asdict

FROM mock_data import MOCK_FILINGS, get_clean_filings, ENTITY_MATCHES
FROM agents.ingestion_agent import run_ingestion, handle_parse_filing_batch
FROM agents.transform_agent import run_transformation
FROM agents.quality_agent import run_quality_checks
FROM agents.reporting_agent import run_reporting
FROM quality_gate import CircuitBreaker, HITLReviewGate


    # … (full source in the desktop HTML) …

    result = run_pipeline("BATCH-2024-0310-DE", "DE", clean, auto_resolve_hitl=True)
    print(f"\nFinal status: {result['status']}")
    print(f"Risk profiles)
    FOR p IN result["state"]["reporting_output"].get("risk_profiles", []):
        print(f"  {p['entity']}: {p['risk_level']} "
              f"({p['active_liens']} active liens)")
```

NODE.JS

```javascript
// coordinator.ts — Multi-Agent Pipeline Orchestrator (Node.js version)
// This is a simplified TypeScript port. The Python version above is the
// primary implementation with full HITL and circuit breaker integration.
// To run this version, install: npm install @anthropic-ai/sdk

IMPORT Anthropic from "@anthropic-ai/sdk";

INTERFACE PipelineState {
  batchId: string;
  sourceState: string;
  stage: string;
  ingestionOutput: Record<string, unknown> | null;
  transformationOutput: Record<string, unknown> | null;
  qualityOutput: Record<string, unknown> | null;

    # … (full source in the desktop HTML) …


  // Stages 1-4 would follow the same pattern as the Python version
  // Each stage processes data and passes results to the next
  state.stage = "complete";
  RETURN { status, state };
}
```

Run the pipeline:

```
python coordinator.py
```

Expected output:

```
============================================================
PIPELINE START — Batch: BATCH-2024-0310-DE, State: DE
Records: 12
============================================================

[1/5] Circuit Breaker Check...
  HEALTHY — Error rate: 0.0%

[2/5] Ingestion Agent...
  Parsed: 12/12

[3/5] Transformation Agent...
  Transformed: 12 records

  HITL GATE: 2 conflicts need review

============================================================
HITL REVIEW REQUIRED — Filing: 2024-0099200
  Candidate A: Acme Logistics LLC (ID: ENT-00234)
  Candidate B: ACME LOGISTICS, L.L.C. (ID: ENT-00234-ALT)
  Match Confidence: 76%
  [AUTO-RESOLVE] Selecting option 1: Merge to A
  Decision recorded: merge_to_a
============================================================

============================================================
HITL REVIEW REQUIRED — Filing: 2024-0099950
  Candidate A: Acme Logistics LLC (ID: ENT-00234)
  Candidate B: Acme  Logistics   LLC (ID: ENT-09950)
  Match Confidence: 68%
  [AUTO-RESOLVE] Selecting option 1: Merge to A
  Decision recorded: merge_to_a
============================================================

[4/5] Quality Agent...
  Quality Grade: A (Score: 100.0%)

[5/5] Reporting Agent...
  Entities profiled: 8

============================================================
PIPELINE COMPLETE — Batch: BATCH-2024-0310-DE
  Quality: A | Entities: 8 | HITL Reviews: 2
============================================================

Final status: complete
Risk profiles:
  ACME LOGISTICS LLC: MEDIUM (2 active liens)
  BuildRight Construction Inc: MEDIUM (2 active liens)
  Acme Logistics LLC: MEDIUM (2 active liens)
  Pacific Coast Distributors Inc: MEDIUM (2 active liens)
  ...
```

What Just Happened?

You ran the complete 4-agent pipeline end-to-end. The coordinator: (1) checked the circuit breaker (healthy for clean data), (2) parsed 12 filings through ingestion, (3) normalized names and classified collateral in transformation, (4) detected 2 entity conflicts below 80% confidence and auto-resolved them via HITL, (5) ran quality checks (grade A), and (6) generated risk profiles for 8 entities. The pipeline passed structured state between every stage — no agent needed to know about the others.

Step 8: Create the Test Suite (test_pipeline.py)

What & Why: Testing is critical for multi-agent systems because failures can be subtle — an agent might produce output that looks right but contains incorrect entity resolutions. This test file covers 5 scenarios: happy path, HITL trigger, circuit breaker trip, edge case (empty batch), and the circuit breaker threshold boundary.

Create `test_pipeline.py` :

PYTHON

PYTHON

```python
# test_pipeline.py — Test cases for the UCC pipeline
IMPORT json
FROM mock_data import MOCK_FILINGS, get_clean_filings, get_error_filings, compute_error_rate
FROM quality_gate import CircuitBreaker, HITLReviewGate
FROM agents.ingestion_agent import handle_parse_filing_batch
FROM agents.transform_agent import run_transformation, normalize_name, classify_collateral
FROM agents.quality_agent import run_quality_checks
FROM agents.reporting_agent import run_reporting, generate_risk_profile

FUNCTION test_1_happy_path():

    print("\n--- Test 1: Happy Path (clean filings) ---")
    clean = get_clean_filings()
    assert len(clean) == 12, f"Expected 12 clean filings, got {len(clean)}"

    # … (full source in the desktop HTML) …

            print(f"  ERROR — {type(e).__name__}: {e}")
            failed += 1

    print(f"\n{'='*60}")
    print(f"Results: {passed} passed, {failed} failed out of {len(tests)} tests")
    print(f"{'='*60}")
```

Run the tests:

```
python test_pipeline.py
```

Expected output:

```
============================================================
UCC PIPELINE TEST SUITE
============================================================

--- Test 1: Happy Path (clean filings) ---
  PASSED — 8 entities, grade A

--- Test 2: Circuit Breaker Trip ---
  PASSED — Tripped at 20.0% (threshold: 10%)

--- Test 3: HITL Review Trigger ---

============================================================
HITL REVIEW REQUIRED — Filing: 2024-0099200
  ...
  [AUTO-RESOLVE] Selecting option 1: Merge to A
  Decision recorded: merge_to_a
============================================================

============================================================
HITL REVIEW REQUIRED — Filing: 2024-0099950
  ...
  [AUTO-RESOLVE] Selecting option 1: Merge to A
  Decision recorded: merge_to_a
============================================================
  PASSED — 2 conflicts auto-resolved

--- Test 4: Edge Case (empty batch) ---
  PASSED — Empty batch handled without errors

--- Test 5: Entity Name Normalization ---
  PASSED — All normalizations correct

============================================================
Results: 5 passed, 0 failed out of 5 tests
============================================================
```

Checkpoint

All 5 tests should pass. Test 2 confirms the circuit breaker trips at 20% error rate. Test 3 confirms HITL triggers for the two entity matches below 80% confidence. If any test fails, check the error message — the most common issue is a missing import or file.

Testing Guide

TYPE	SCENARIO	EXPECTED BEHAVIOR
Happy	245 DE XML filings, 2.8% error rate	All 4 agents complete, quality scorecard generated, reports produced
Happy	Entity with 76% confidence match	Pipeline pauses, routes to data steward queue
Happy	Steward selects "match_to_ENT-00234"	Pipeline resumes with resolved entity, continues through quality+reporting
Happy	Clean batch with no conflicts	Full pipeline: ingestion → transform → quality → reporting, no HITL
Happy	Batch with mixed active/lapsed filings	Quality agent correctly categorizes, reporting separates active vs lapsed
Edge	State sends CSV instead of expected XML	Ingestion agent detects format, uses correct parser
Edge	Quality agent detects 80% drop in filing count	Flags anomaly for investigation, does not auto-halt
Edge	Collateral description is empty	Transform agent flags as incomplete, quality agent deducts from completeness score
Adversarial	12% parse error rate in batch	Circuit breaker trips, batch quarantined, team alerted
Adversarial	Malformed CSV with injection characters	Parser treats as literal data, does not execute, flags as parse error

Verify Everything Works

Run the complete pipeline end-to-end with a single command:

```
python coordinator.py && python test_pipeline.py
```

Expected final output: The coordinator runs the full pipeline on 12 clean filings, auto-resolves 2 HITL conflicts, and produces risk profiles. The test suite then runs all 5 tests and reports 5 passed, 0 failed.

You can also test the circuit breaker trip by running the pipeline on the full dataset (including bad records):

```
python -c "
FROM coordinator import run_pipeline
FROM mock_data import MOCK_FILINGS
result = run_pipeline('BATCH-BAD', 'DE', MOCK_FILINGS)
print(f'Status: {result[\"status\"]}')
"
```

Expected output:

```
============================================================
PIPELINE START — Batch: BATCH-BAD, State: DE
Records: 15
============================================================

[1/5] Circuit Breaker Check...
  TRIPPED! Error rate 20.0% exceeds 10% threshold.
  Batch BATCH-BAD quarantined. Pipeline halted.
Status: circuit_breaker_tripped
```

CONGRATULATIONS!

You have built a complete multi-agent UCC data pipeline with four specialized agents, a circuit breaker that halts on data quality failures, and human-in-the-loop review for ambiguous entity matches. This is the same architecture used in production data engineering systems — the only differences in a real deployment would be connecting to actual state SOS data feeds and a BigQuery Gold layer instead of mock data.

Troubleshooting

Common Errors

IMPORT ERRORS

`ModuleNotFoundError: No module named 'anthropic'` — Your virtual environment is not activated. Run `source venv/bin/activate` (Unix) or `venv\Scripts\activate` (Windows).

`ModuleNotFoundError: No module named 'agents'` — You are running the script from the wrong directory. Make sure you are inside `capstone-4-multi-agent-ucc/`.

`ImportError: cannot import name 'run_ingestion' from 'agents'` — The `agents/__init__.py` file is missing or one of the agent files has not been created yet. Complete all steps before running the coordinator.

API ERRORS

`AuthenticationError: 401` — Your `ANTHROPIC_API_KEY` is not set or is invalid. Run `echo $ANTHROPIC_API_KEY` (Unix) or `echo %ANTHROPIC_API_KEY%` (Windows) to verify.

`RateLimitError: 429` — You are sending too many requests. Wait a few seconds and retry. For testing, the coordinator uses mock tool handlers so API calls are minimal.

TEST FAILURES

Test 5 (normalization) fails — Check that your `classify_collateral` function checks keywords in the correct order. The function returns categories in the order they are matched, which depends on dictionary iteration order in Python 3.7+.

Circuit breaker doesn't trip — Ensure your `mock_data.py` has exactly 3 filings with `"parse_error": True`. The error rate should be 3/15 = 20%.

HITL doesn't trigger — Ensure your `ENTITY_MATCHES` list in `mock_data.py` has at least 2 entries with `"confidence"` below 0.80.

Compliance & Regulatory Notes

DATA PIPELINE REGULATIONS

PII redaction: The Reporting Agent must redact personally identifiable information before generating public-facing reports. Individual names, addresses, and Social Security numbers (if present in collateral descriptions) must be masked.

FCRA compliance: If pipeline output is used for credit decisions, accuracy requirements under the Fair Credit Reporting Act apply. The circuit breaker and quality checks are compliance mechanisms — they prevent inaccurate data from reaching decision-making systems.

Audit trail: Every pipeline run must log: batch ID, source state, records processed, error count, HITL decisions, quality scores, and output file locations. This enables regulatory audits and dispute resolution.